Utility Routines Reference Manual

Модули `utility_routines`, `avproto`, `cli_routines`, `starlet`

Справочное руководство программиста

Стиль оформления — VSI OpenVMS RTL (LIB$) Reference

Оглавление

Chapter 1. Utility Routines (UTIL$) Reference

- $LOG
- __util$deflog
- __util$setlogfd
- __util$putmsg_fao
- __util$log2buf
- __util$out
- $TRACE
- $DUMPHEX
- __util$faohex
- __util$str2asc
- __util$asc2str
- __util$cmpasc
- __util$getparams
- __util$readconfig
- __util$showparams
- __util$bin2hex
- __util$hex2bin
- __util$uuid2str
- __util$crc32c
- $INSQTAIL
- $REMQHEAD
- __util$movc5
- __util$strstr
- __util$pattern_match

Chapter 2. AV Protocol Routines (avproto) Reference

- __tlv$hset
- __tlv$hget
- __tlv$encode_tag
- __tlv$decode_tag
- __tlv$put
- __tlv$get
- __tlv$lookup
- __tlv$dump

Chapter 3. Command Language Interface (CLI$) Reference

- __cli$parse
- __cli$dispatch
- __cli$get_value
- __cli$present
- __cli$cleanup
- __cli$show_verbs
- __cli$help

Chapter 4. Time and FAO Services (starlet) Reference

```
__util$fao
__util$faol
__util$bintim
__util$gettim
__util$asctim
__util$add_times
__util$sub_times
```

Chapter 5. Programming Examples

Глава 1. Справочник рутин утилит (UTIL$)

Эта глава описывает вызываемые рутины и макросы модуля `utility_routines`. Рутины следуют соглашению OpenVMS о кодах-состояниях: младший бит возвращаемого статуса установлен при успехе (`STS$K_SUCCESS`, `STS$K_INFO`) и сброшен при неудаче (`STS$K_WARN`, `STS$K_ERROR`, `STS$K_FATAL`). Перед включением `utility_routines.h` определите `__MODULE__` и, опционально, `__FAC__`.

$LOG

Макрос Write Formatted Log Record форматирует сообщение по printf-подобной форматной строке и записывает его - с меткой времени, уровнем важности и префиксом местоположения - в текущий приёмник журнала.

Формат `$LOG severity, format [, argument, ...]`

Возврат `unsigned int` (write only) — Переданное значение **severity** - чтобы макрос можно было использовать прямо в операторе `return`.

Параметр	Тип	Доступ	Описание
`severity`	`unsigned int`	read only	Уровень важности записи: одно из `STS$K_SUCCESS`, `STS$K_INFO`, `STS$K_WARN`, `STS$K_ERROR` или `STS$K_FATAL`. Соответствующая буква (S/I/W/E/F) выводится в префиксе записи.
`format`	`const char *`	read only	printf-подобная нуль-терминированная форматная строка. Помимо стандартных преобразований, специальное `%C` выводит однобуквенный код важности.
`argument`	`...`	read only	Необязательные аргументы, потребляемые преобразованиями в **format**.

$LOG - обычная точка входа для диагностики. Он разворачивается в вызов `__util$logd()`, автоматически подставляя facility (`__FAC__`), модуль (`__MODULE__`), функцию и номер строки места вызова. Каждая запись предваряется префиксом `DD-MM-YYYY HH:MM:SS.msec <pid/tid> [module\function:line] %FAC-S:` и пишется в приёмник, выбранный `__util$deflog()` (по умолчанию - стандартный вывод).

Вызывающий обязан определить `__MODULE__` (и опционально `__FAC__`) до включения `utility_routines.h`.

Условные значения

`STS$K_*` — Рутина возвращает аргумент **severity** без изменений.

__util$deflog

Рутина Define Log Destination перенаправляет последующий вывод журнала в файл и, опционально, на удалённый SYSLOG-хост.

Формат `int __util$deflog (logfile, sloghost)`

Возврат `int` (write only) — Код-состояние; при успехе установлен младший бит.

Параметр	Тип	Доступ	Описание
`logfile`	`const char *`	read only	Нуль-терминированный путь лог-файла для дозаписи, или `NULL`, чтобы оставить стандартный вывод. Файл открывается с `O_CLOEXEC`.
`sloghost`	`const char *`	read only	Нуль-терминированный IPv4-адрес SYSLOG-сервера, или `NULL`, если не нужен.

Вызывается один раз при старте, до первого `$LOG`. Если задан SYSLOG-хост, записи дополнительно пересылаются UDP-датаграммами на стандартный порт syslog.

Условные значения

`STS$K_SUCCESS`	Рутина успешно завершена.
`STS$K_ERROR`	Лог-файл не удалось открыть; причину указывает `errno`.

__util$setlogfd

Рутина Set Log Descriptor направляет журналирование на уже открытый дескриптор, не открывая файл и не захватывая стандартные потоки.

Формат `int __util$setlogfd (a_fd)`

Возврат `int` (write only) — Код-состояние; при успехе установлен младший бит.

Параметр	Тип	Доступ	Описание
`a_fd`	`int`	read only	Уже открытый дескриптор для приёма записей журнала. Отрицательное значение отвергается.

Дополняет `__util$deflog()`, которая принимает имя файла и захватывает и стандартный вывод, и стандартную ошибку через `dup2()` - подходит для демона, но не для утилиты, чей стандартный вывод несёт данные, ради которых её запускали. Такая утилита один раз вызывает `__util$setlogfd(STDERR_FILENO)`, направляя записи `$LOG` / `$TRACE` в стандартную ошибку и оставляя стандартный вывод чистым.

Условные значения

`STS$K_SUCCESS`	Рутина успешно завершена.
`STS$K_ERROR`	Дескриптор отрицательный; текущий приёмник оставлен без изменений.

__util$putmsg_fao

Рутина Put Message (FAO) сигнализирует сообщение из зарегистрированного каталога, принимая параметры через службу форматирования FAO, а не через printf-формат.

Формат `unsigned __util$putmsg_fao (sts, prmcnt, ...)`

Возврат `unsigned` (write only) — Аргумент , возвращаемый без изменений, чтобы рутину можно было использовать как операнд return.

Параметр	Тип	Доступ	Описание
`sts`	`unsigned`	read only	Сигнализируемое условное значение. Его фасилити и важность обрамляют запись; номер сообщения выбирает текст из каталога.
`prmcnt`	`int`	read only	Число следующих далее FAO-параметров, от 0 до 16. Проставляется за вызывающего макросом `$PUTMSG_FAO()` - см. ниже.
`...`	`fao_prm_t`	read only	FAO-параметры, каждый - 64-битный `fao_prm_t`, подставляемые в FAO-строку записи (`!AS`, `!UL`, `!XL`, `!UQ` и т. п.).

Сигнализирует сообщение каталога, как `__util$putmsg()`, но текст записи - это FAO-строка, форматируемая через `__util$faol()`, поэтому сообщение может нести типизированные директивы - `!UL`, `!XL`, `!AS`, `!UQ` - вместо printf-формата. Каталог - тот, что зарегистрирован `__util$inimsg()`; для значения без записи выдаётся диагностика по умолчанию. Запись сопоставляется по фасилити и номеру сообщения, биты severity игнорируются, поэтому один статус может сигнализироваться как `INFO` в одном месте и как `WARNING` в другом из одной записи. Найденная запись оформляется в OpenVMS-виде `%<facility>-<S>-<text>`: имя фасилити из дескриптора, с которым она зарегистрирована (или `NONAME`), и буква severity из `$SEV` статуса. Статус также возвращается, так что его severity управляет логикой вызывающего. Запись обрамляется меткой времени, идентификатором процесса и, в отладочной сборке, модулем и строкой, и пишется в приёмник, выбранный `__util$deflog()` / `__util$setlogfd()`.

Параметры идут через varargs и читаются как 64-битный `fao_prm_t`, поэтому каждый должен быть приведён к этому типу - непреобразованное 32-битное значение есть неопределённое поведение. Макрос `$PUTMSG_FAO()` выполняет приведение за вызывающего и выводит счётчик из списка, так что значение пишется просто: `$PUTMSG_FAO(sts, l_seq, l_disp)`. Счётчик, написанный вручную, может разойтись со списком; выведенный из него - не может.

Условные значения

`(возвращает sts)` Рутина возвращает свой аргумент без изменений.

__util$log2buf

Рутина Format Log Record into Buffer форматирует запись журнала так же, как $LOG, но помещает её в предоставленный вызывающим буфер, а не в приёмник журнала.

Формат `unsigned __util$log2buf (out, outsz, outlen, fac, severity, format [, argument, ...])`

Возврат `unsigned int` (write only) — Переданное значение **severity**.

Параметр	Тип	Доступ	Описание
`out`	`void *`	write only	Адрес выходного буфера.
`outsz`	`int`	read only	Размер **out** в байтах.
`outlen`	`int *`	write only	Принимает число байт, записанных в **out**.
`fac`	`const char *`	read only	Имя facility для префикса.
`severity`	`unsigned int`	read only	Важность записи.
`format`	`const char *`	read only	printf-подобная форматная строка.
`argument`	`...`	read only	Необязательные аргументы формата.

Полезна, когда форматированный текст нужен как строка (для поля протокола, элемента интерфейса и т.п.), а не для записи в журнал.

Условные значения

`STS$K_*`	Рутина возвращает аргумент **severity**.

__util$out

Рутина Write Plain Record записывает printf-форматированную строку в приёмник журнала без префикса метка-времени/важность.

Формат `unsigned __util$out (format [, argument, ...])`

Возврат `unsigned int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`format`	`char *`	read only	printf-подобная форматная строка.
`argument`	`...`	read only	Необязательные аргументы формата.

Предназначена для баннеров, таблиц и другого вывода, который не должен нести диагностический префикс.

Условные значения

`STS$K_SUCCESS`	Рутина успешно завершена.

$TRACE

Макрос Write Trace Record выводит диагностическую трассировочную запись в отладочной сборке и компилируется в ничто, если определён NDEBUG.

Формат `$TRACE format [, argument, ...]`

Возврат Нет.

Параметр	Тип	Доступ	Описание
`format`	`const char *`	read only	printf-подобная форматная строка.
`argument`	`...`	read only	Необязательные аргументы формата.

$TRACE разворачивается в вызов `__util$trace()`, несущий модуль, функцию и строку места вызова. При компиляции с `-DNDEBUG` макрос становится пустышкой, поэтому точки трассировки можно оставлять в промышленном коде без накладных расходов. Родственный `$IFTRACE(cond, ...)` выводит запись только когда **cond** истинно.

$DUMPHEX

Макрос Dump Buffer in Hexadecimal записывает форматированный hex/ASCII-дамп области памяти в приёмник журнала.

Формат `$DUMPHEX buffer, length`

Возврат Нет.

Параметр	Тип	Доступ	Описание
`buffer`	`const void *`	read only	Адрес дампируемой области.
`length`	`unsigned short`	read only	Число байт для дампа.

Разворачивается в `__util$dumphex()`, подставляя функцию и строку места вызова. Вывод располагается по шестнадцать октетов в строке в виде `offset: hex ... ascii`.

__util$faohex

Рутина Format One Hexadecimal Dump Line форматирует до шестнадцати октетов в одну строку hex-и-ASCII дампа.

Формат `int __util$faohex (src, srclen, out, outsz, outlen)`

Возврат `int` (write only) — Число записанных байт, или 0 при ошибке.

Параметр	Тип	Доступ	Описание
`src`	`const void *`	read only	Адрес форматируемых октетов.
`srclen`	`unsigned short`	read only	Число октетов; значения больше 16 усекаются до 16.
`out`	`char *`	write only	Адрес выходного буфера.
`outsz`	`unsigned short`	read only	Размер **out**; должен быть не меньше 67 байт, иначе рутина вернёт 0.
`outlen`	`unsigned short *`	write only	Принимает число байт, записанных в **out**.

Формирует одну строку вида `XX XX ... XX ....ascii.....` Неиспользованные колонки дополняются пробелами, непечатаемые октеты показываются точкой.

Условные значения

`>= 0` Число записанных байт; 0 означает, что **outsz** слишком мал.

__util$str2asc

Рутина Copy String to Counted String копирует нуль-терминированную строку в ASCIC-контейнер (со счётчиком длины).

Формат `int __util$str2asc (src, dst)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`src`	`const char *`	read only	Нуль-терминированная строка-источник.
`dst`	`ASC *`	write only	ASCIC-контейнер назначения.

Длина берётся из источника до `ASC$K_SZ` (255) октетов. Обратное преобразование выполняет `__util$asc2str()`.

Условные значения

`STS$K_SUCCESS`	Рутина успешно завершена.

__util$asc2str

Рутина Copy Counted String to String копирует ASCIC-контейнер в нуль-терминированную строку.

Формат `int __util$asc2str (src, dst)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`src`	`ASC *`	read only	ASCIC-контейнер источника.
`dst`	`char *`	write only	Буфер назначения; должен вмещать `src->len + 1` байт.

Добавляется завершающий NUL.

Условные значения

`STS$K_SUCCESS`	Рутина успешно завершена.

__util$cmpasc

Рутина Compare Counted Strings сравнивает два ASCIC-контейнера на равенство.

Формат `int __util$cmpasc (a, b)`

Возврат `int` (write only) — Ненулевое значение, если строки равны.

Параметр	Тип	Доступ	Описание
`a`	`ASC *`	read only	Первая ASCIC-строка.
`b`	`ASC *`	read only	Вторая ASCIC-строка.

Сначала сравниваются длины, затем байты. Родственная `__util$cmpasc_blind()` выполняет сравнение без учёта регистра.

__util$getparams

Рутина Parse Command-Line Options заполняет таблицу типизированных дескрипторов опций из вектора аргументов программы.

Формат `int __util$getparams (argc, argv, opts)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`argc`	`int`	read only	Число аргументов, как передано в main().
`argv`	`char *[]`	read only	Вектор аргументов.
`opts`	`const OPTS *`	modify	Адрес массива дескрипторов `OPTS`, оканчивающегося `OPTS_NULL`. Каждый дескриптор задаёт имя опции, указывает на приёмную переменную, её размер и тип `OPTS$K_*`.

Принимает опции в виде `-name=value`, `/name=value` или `--name=value`. Имена опций можно сокращать до любого однозначного префикса; аргумент длиннее имени опции ей не соответствует. Значения преобразуются согласно типу дескриптора (`OPTS$K_INT`, `OPTS$K_STR`, `OPTS$K_OPT`, `OPTS$K_PORT`, `OPTS$K_IP4`, `OPTS$K_CONF`, `OPTS$K_PWD`). Дескриптор типа `OPTS$K_CONF` вызывает немедленное чтение указанного конфиг-файла через `__util$readconfig()`.

Условные значения

`STS$K_SUCCESS`	Все распознанные опции разобраны.
`STS$K_ERROR`	Значение опции не удалось преобразовать, либо отсутствовала обязательная опция.

__util$readconfig

Рутина Read Configuration File заполняет таблицу типизированных дескрипторов опций из текстового конфигурационного файла.

Формат `int __util$readconfig (fconf, opts)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`fconf`	`const char *`	read only	Нуль-терминированный путь конфиг-файла. Пустой или NULL-путь - ошибка.
`opts`	`OPTS *`	modify	Адрес таблицы `OPTS`, оканчивающейся `OPTS_NULL`, как для `__util$getparams()`.

Файл содержит по одной опции в строке в том же виде `-name=value`, что и в командной строке; текст после `!` считается комментарием. Одну таблицу опций можно использовать как для командной строки, так и для конфиг-файла.

Условные значения

`STS$K_SUCCESS`	Файл прочитан и разобран.
`STS$K_ERROR`	Имя файла пустое, файл не удалось открыть, либо значение недопустимо.

__util$showparams

Рутина Display Option Values записывает текущее значение каждого дескриптора таблицы опций в журнал.

Формат `int __util$showparams (opts)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`opts`	`const OPTS *`	read only	Адрес таблицы `OPTS`, оканчивающейся `OPTS_NULL`.

Дескрипторы паролей (`OPTS$K_PWD`) показываются замаскированными.

Условные значения

`STS$K_SUCCESS`	Рутина успешно завершена.

__util$bin2hex

Рутина Convert Binary to Hexadecimal преобразует последовательность октетов в строчную шестнадцатеричную строку.

Формат `int __util$bin2hex (src, dst, srclen)`

Возврат `int` (write only) — Длина шестнадцатеричной строки, `srclen * 2`.

Параметр	Тип	Доступ	Описание
`src`	`const void *`	read only	Адрес октетов-источника.
`dst`	`void *`	write only	Буфер назначения; должен вмещать `srclen * 2 + 1` байт. Записывается завершающий NUL.
`srclen`	`unsigned short`	read only	Число октетов-источника.

Удобный макрос `$BIN2HEX(s, d, l)` подставляет приведения типов. Обратное преобразование выполняет `__util$hex2bin()`.

Условные значения

`>= 0`	Число записанных шестнадцатеричных символов.

__util$hex2bin

Рутина Convert Hexadecimal to Binary преобразует шестнадцатеричную строку обратно в октеты.

Формат `int __util$hex2bin (src, dst, srchexlen)`

Возврат `int` (write only) — Число записанных октетов.

Параметр	Тип	Доступ	Описание
`src`	`const void *`	read only	Адрес hex-строки.
`dst`	`void *`	write only	Буфер октетов назначения.
`srchexlen`	`unsigned short`	read only	Число шестнадцатеричных символов для преобразования (два на выходной октет).

Принимаются как заглавные, так и строчные шестнадцатеричные цифры.

Условные значения

`>= 0`	Число записанных октетов.

__util$uuid2str

Рутина Convert UUID to String форматирует 16-октетный UUID в его каноническое текстовое представление.

Формат `int __util$uuid2str (uuid, buf, bufsz)`

Возврат `int` (write only) — Длина форматированной строки.

Параметр	Тип	Доступ	Описание
`uuid`	`void *`	read only	Адрес 16-октетного UUID.
`buf`	`char *`	write only	Буфер назначения; достаточно 37 байт.
`bufsz`	`int`	read only	Размер **buf** в байтах.

Результат имеет вид `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`.

Условные значения

`>= 0`	Число записанных символов.

__util$crc32c

Рутина Compute CRC-32 Checksum вычисляет 32-битную циклическую избыточную проверку по области памяти.

Формат `unsigned __util$crc32c (crc, buf, buflen)`

Возврат `unsigned int` (write only) — 32-битная контрольная сумма.

Параметр	Тип	Доступ	Описание
`crc`	`unsigned int`	read only	Начальное значение. Передайте 0, чтобы начать новую сумму; передайте результат предыдущего вызова, чтобы продолжить инкрементальное вычисление.
`buf`	`const void *`	read only	Адрес данных.
`buflen`	`size_t`	read only	Число октетов.

Несмотря на имя, эта рутина вычисляет стандартный CRC-32 IEEE 802.3 / zlib (отражённый полином 0xEDB88320), а **не** Castagnoli CRC-32C; контрольное значение `__util$crc32c(0, "123456789", 9)` равно `0xCBF43926`. На платформах ARMv8 с расширением CRC используются аппаратные инструкции `crc32`; в остальных случаях - переносимая реализация slice-by-eight. Результат одинаков на всех платформах и порядках байт.

Условные значения

`longword`	Значение контрольной суммы (эта рутина не использует соглашение о кодах-состояниях).

$INSQTAIL

Макрос Insert Entry at Tail of Queue связывает элемент с хвостом двусвязной очереди под защитой spinlock.

Формат `$INSQTAIL queue, entry, count`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
queue	`__QUEUE *`	modify	Адрес заголовка очереди.
entry	`void *`	modify	Адрес элемента, первым членом которого обязан быть `ENTRY`. Перед первой вставкой элемент должен быть обнулён.
count	`unsigned *`	write only	Принимает число элементов в очереди до вставки.

Разворачивается в `__util$insqtail()`. `$INSQHEAD` вставляет в голову. Структура элемента должна начинаться с члена `ENTRY`; заголовок очереди - `__QUEUE`, инициализированный через `QUEUE_INITIALIZER`.

Условные значения

`STS$K_SUCCESS`	Элемент вставлен.
`UTIL$S_INQUE`	Элемент уже связан с другой очередью.
`UTIL$S_INVARG`	Передан NULL-аргумент.

$REMQHEAD

Макрос Remove Entry from Head of Queue отвязывает элемент в голове двусвязной очереди.

Формат `$REMQHEAD queue, entry, count`

Возврат `int` (write only) — Код-состояние (см. ниже).

Параметр	Тип	Доступ	Описание
queue	`__QUEUE *`	modify	Адрес заголовка очереди.
entry	`void **`	write only	Принимает адрес удалённого элемента. **Не** записывается, когда очередь пуста.
count	`unsigned *`	write only	Принимает число элементов в очереди до удаления.

Разворачивается в `__util$remqhead()`; `$REMQTAIL` удаляет из хвоста. Рутина возвращает `STS$K_SUCCESS` даже на пустой очереди; в этом случае **entry** не изменяется, а **count** равен нулю. Поэтому цикл извлечения обязан проверять **count**, чтобы обнаружить пустоту, иначе он не завершится.

Условные значения

`STS$K_SUCCESS`	Операция выполнена. Элемент удалён только если **count** был ненулевым.
`STS$K_ERROR`	Передан NULL-аргумент, либо не удалось взять блокировку.

__util$movc5

Рутина Move Characters with Fill копирует область-источник в область-назначение, дополняя октетом-заполнителем или усекая по необходимости. Повторяет примитив OpenVMS LIB$MOVC5.

Формат `int __util$movc5 (srclen, source, fill, dstlen, dest)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`srclen`	`unsigned short *`	read only	Адрес длины источника в октетах.
`source`	`char *`	read only	Адрес области-источника.
`fill`	`unsigned char *`	read only	Адрес октета-заполнителя, используемого, когда назначение длиннее источника.
`dstlen`	`unsigned short *`	read only	Адрес длины назначения в октетах.
`dest`	`char *`	write only	Адрес области-назначения.

Если назначение короче источника, октеты источника со старшими адресами не перемещаются. Если длиннее - остаток заполняется октетом-заполнителем.

Условные значения

`STS$K_SUCCESS` Рутина успешно завершена.

__util$strstr

Рутина Locate Substring by Length ищет в области со счётчиком первое вхождение образца со счётчиком, не полагаясь на NUL-терминацию.

Формат `char * __util$strstr (s1, s1len, s2, s2len)`

Возврат `char *` (write only) — Адрес первого вхождения, или `NULL`, если образец не найден.

Параметр	Тип	Доступ	Описание
`s1`	`char *`	read only	Адрес области поиска.
`s1len`	`size_t`	read only	Длина области.
`s2`	`char *`	read only	Адрес образца.
`s2len`	`size_t`	read only	Длина образца.

Оба операнда могут содержать встроенные октеты NUL.

Условные значения

`address` Указатель внутрь **s1**, или `NULL`.

__util$pattern_match

Рутина Match String Against Pattern проверяет, соответствует ли строка шаблону с подстановочными знаками.

Формат `int __util$pattern_match (str, pattern)`

Возврат `int` (write only) — Ненулевое значение, если строка соответствует.

Параметр	Тип	Доступ	Описание
`str`	`char *`	read only	Нуль-терминированная проверяемая строка.
`pattern`	`char *`	read only	Нуль-терминированный шаблон. `*` соответствует любой последовательности символов, а `%` - ровно одному символу.

Сравнение с учётом регистра.

Условные значения

`1`	Строка соответствует шаблону.
`0`	Строка не соответствует.

Глава 2. Справочник рутин протокола avproto

Эта глава описывает рутины модуля `avproto` - компактного двоичного протокола из фиксированного 20-октетного заголовка и переменного числа записей Tag-Length-Value (TLV). Все многооктетные поля хранятся в сетевом порядке байт. Пакет (PDU) адресуется через тип `AVPROTO_PDU`; рутины носят префикс `__tlv$`. Типы тегов - константы `TAG$K_*` (`BBLOCK`=0, `WORD`=3, `LONGWORD`=4, `QWORD`=5, `UUID`=8).

__tlv$hset

Рутина Set PDU Header инициализирует фиксированный заголовок пакета (PDU) и сбрасывает длину его полезной нагрузки в ноль.

Формат `int __tlv$hset (pdu, sig, u_csr, u_seq)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`pdu`	`void *`	write only	Адрес буфера PDU.
`sig`	`const unsigned char *`	read only	Восьмиоктетная сигнатура протокола, записываемая в заголовок.
`u_csr`	`unsigned int`	read only	Значение регистра команды/состояния.
`u_seq`	`unsigned int`	read only	Порядковый номер PDU.

Должна вызываться до первого `__tlv$put()`, поскольку обнуляет длину полезной нагрузки. Все поля заголовка хранятся в сетевом порядке байт.

Условные значения

`STS$K_SUCCESS`	Рутина успешно завершена.

__tlv$hget

Рутина Get PDU Header проверяет сигнатуру PDU и извлекает поля заголовка.

Формат `int __tlv$hget (pdu, sig, u_len, u_csr, u_seq)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`pdu`	`void *`	read only	Адрес принятого PDU.
`sig`	`const unsigned char *`	read only	Ожидаемая восьмиоктетная сигнатура; несовпадение - ошибка.
`u_len`	`unsigned int *`	write only	Принимает длину полезной нагрузки, объявленную в заголовке.
`u_csr`	`unsigned int *`	write only	Принимает регистр команды/состояния.
`u_seq`	`unsigned int *`	write only	Принимает порядковый номер.

Все возвращаемые поля преобразуются из сетевого в хостовый порядок байт.

Условные значения

`STS$K_SUCCESS`	Сигнатура совпала, поля извлечены.
`STS$K_ERROR`	Сигнатура не совпала.

__tlv$encode_tag

Рутина Encode TLV Tag упаковывает тип тега и идентификатор тега в шестнадцатибитное поле тега TLV.

Формат `unsigned short __tlv$encode_tag (v_type, v_tag)`

Возврат `unsigned short` (write only) — Упакованное поле `w_tag` в сетевом порядке байт.

Параметр	Тип	Доступ	Описание
`v_type`	`unsigned short`	read only	Тип тега; одна из констант `TAG$K_*` (четыре бита).
`v_tag`	`unsigned short`	read only	Прикладной идентификатор тега (двенадцать бит).

Компоновка - `[ type : 4 | id : 12 ]`, возвращается в сетевом порядке байт.

__tlv$decode_tag

Рутина Decode TLV Tag извлекает тип тега и идентификатор тега из поля тега TLV.

Формат `unsigned short __tlv$decode_tag (w_tag, v_type, v_tag)`

Возврат `unsigned short` (write only) — Поле тега в порядке байт хоста.

Параметр	Тип	Доступ	Описание
`w_tag`	`unsigned short`	read only	Поле `w_tag` в сетевом порядке байт.
`v_type`	`unsigned short *`	write only	Принимает тип тега.
`v_tag`	`unsigned short *`	write only	Принимает идентификатор тега.

Старшие четыре бита дают тип, младшие двенадцать - идентификатор.

__tlv$put

Рутина Append TLV добавляет одну запись Tag-Length-Value к PDU.

Формат `int __tlv$put (pdu, pdusz, v_tag, v_type, val, valsz)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`pdu`	`void *`	modify	Адрес буфера PDU.
`pdusz`	`unsigned short`	read only	Полный размер буфера PDU в октетах.
`v_tag`	`unsigned int`	read only	Прикладной идентификатор тега.
`v_type`	`unsigned int`	read only	Тип тега: `TAG$K_BBLOCK`, `TAG$K_WORD`, `TAG$K_LONGWORD`, `TAG$K_QWORD` или `TAG$K_UUID`.
`val`	`void *`	read only	Адрес сохраняемого значения.
`valsz`	`unsigned int`	read only	Размер значения в октетах. Для целочисленных типов фиксированной ширины может быть нулём.

Значения word, longword и quadword хранятся в сетевом порядке байт. Рутина дописывает после текущей полезной нагрузки и обновляет длину в заголовке. Она завершается неудачей, если значение не помещается в оставшемся месте буфера.

Условные значения

`STS$K_SUCCESS`	Запись добавлена.
`STS$K_ERROR`	Буфер слишком мал, либо заголовок объявляет повреждённую длину.

__tlv$get

Рутина Extract TLV Value находит TLV по тегу и копирует его значение в буфер вызывающего.

Формат `int __tlv$get (pdu, pdusz, context, v_tag, v_type, val, valsz)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`pdu`	`void *`	read only	Адрес принятого PDU.
`pdusz`	`unsigned int`	read only	**Реальный** размер области PDU, то есть число фактически принятых октетов. Каждый TLV проверяется по этой границе, поэтому повреждённый или враждебный PDU не может вызвать выход за пределы. Не передавайте сюда длину из заголовка; ей нет доверия.
`context`	`int *`	modify	Контекст итерации. Установите -1 для первого вызова; при повторном использовании между вызовами поиск продвигается вперёд по PDU.
`v_tag`	`unsigned int`	read only	Идентификатор искомого тега.
`v_type`	`unsigned int *`	write only	Принимает тип тега найденной записи.
`val`	`void *`	write only	Адрес буфера для приёма значения.
`valsz`	`unsigned int *`	modify	На входе - размер **val**; на возврате - число скопированных октетов. Для типов переменной длины (BBLOCK, UUID) этот аргумент обязателен; его отсутствие - ошибка.

Значения word, longword и quadword преобразуются из сетевого в хостовый порядок байт. Когда нужен указатель на значение без копирования, используйте `__tlv$lookup()`.

Условные значения

`STS$K_SUCCESS`	Запись найдена, её значение скопировано.
`STS$K_WARN`	Запись с заданным тегом не найдена.
`STS$K_ERROR`	Назначение слишком мало, либо **valsz** опущен для типа переменной длины.

__tlv$lookup

Рутина Locate TLV Value находит TLV по тегу и возвращает указатель на его значение внутри PDU, без копирования.

Формат `int __tlv$lookup (pdu, pdusz, context, v_tag, v_type, val, valsz)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`pdu`	`void *`	read only	Адрес принятого PDU.
`pdusz`	`unsigned int`	read only	Реальный размер области PDU, как для `__tlv$get()`.
`context`	`int *`	modify	Контекст итерации; -1 при первом вызове.
`v_tag`	`unsigned int`	read only	Идентификатор искомого тега.
`v_type`	`unsigned int *`	write only	Принимает тип тега.
`val`	`void **`	write only	Принимает указатель на значение внутри PDU. Данные **не** преобразуются по порядку байт.
`valsz`	`unsigned int *`	write only	Принимает длину значения.

Возвращаемый указатель ссылается на буфер PDU вызывающего и действителен только пока действителен этот буфер. Подходит для больших блоков октетов, которые не нужно копировать.

Условные значения

`STS$K_SUCCESS`	Запись найдена.
`STS$K_WARN`	Запись с заданным тегом не найдена.

__tlv$dump

Рутина Dump PDU записывает в журнал человекочитаемый список всех TLV в PDU.

Формат `void __tlv$dump (pdu, pdusz)`

Возврат Нет.

Параметр	Тип	Доступ	Описание
`pdu`	`void *`	read only	Адрес PDU.
`pdusz`	`unsigned int`	read only	Реальный размер области PDU, как для `__tlv$get()`.

Каждая запись выводится с тегом, типом, длиной и шестнадцатеричным дампом значения. Предназначено для отладки.

Глава 3. Справочник интерфейса командного языка (CLI$)

Эта глава описывает вызываемые рутины модуля `cli_routines` - разборщика командной строки в стиле DCL. Команда имеет грамматику `verb [p1 p2 ... p8] [/qualifier[=value] ...]`. Рутины носят префикс `__cli$`; константы `CLI$K_*` сохраняют префикс `CLI$`. Глаголы описываются таблицей записей `CLI_VERB`; каждый параметр и квалификатор - `CLI_PQDESC` с типом значения. Каждое значение проверяется по своему типу:

`CLI$K_FILE = 1`	Спецификация файла (любая строка).
`CLI$K_DATE = 2`	Строка даты/времени.
`CLI$K_NUM = 3`	Число (8-, 10- или 16-ричное).
`CLI$K_IPV4 = 4`	Адрес IPv4.
`CLI$K_IPV6 = 5`	Адрес IPv6.
`CLI$K_OPT = 7`	Флаг (без значения).
`CLI$K_QSTRING = 8`	Строка в кавычках.
`CLI$K_UUID = 9`	Канонический UUID.
`CLI$K_DEVICE = 0xA`	Устройство в `/dev/`.
`CLI$K_KWD = 0xB`	Одно из списка ключевых слов.
`CLI$K_TIME = 0xC`	Абсолютное или дельта-время, проверяется через starlet `__util$bintim`.
`CLI$K_MAC = 0xD`	MAC-адрес: `aa:bb:cc:dd:ee:ff` или `aa-bb-...`

Опции обработки - `CLI$M_OPTRACE` (трассировать разбор) и `CLI$M_OPSIGNAL` (выводить диагностику на каждую ошибку). Дескриптор квалификатора может нести `CLI$M_NEGATABLE` (принимает `/NO<name>`), `CLI$M_LIST` (список `/Q=(a,b,c)`) или `CLI$M_REQUIRED` (обязателен); непустое поле `defval` задаёт значение по умолчанию. Встроенный `/HELP` (а также `-h`/`--help`) печатает использование через `__cli$help`. Рутины следуют соглашению о кодах-состояниях (младший бит установлен при успехе).

__cli$parse

Рутина Parse Command Line сопоставляет аргументы программы с таблицей определения команд, проверяя глаголы, позиционные параметры и квалификаторы, и строит контекст с результатом разбора.

Формат `int __cli$parse (verbs, opts, argc, argv, clictx)`

Возврат `int` (write only) — Код-состояние; при успехе установлен младший бит.

Параметр	Тип	Доступ	Описание
verbs	CLI_VERB *	read only	Адрес таблицы определения команд: массив дескрипторов CLI_VERB, каждый из которых именует глагол и указывает на его таблицы параметров и квалификаторов, рутину-действие и необязательную цепочку под-глаголов.
opts	int	read only	Опции обработки: CLI$M_OPTRACE трассирует разбор, CLI$M_OPSIGNAL выводит диагностику на каждую синтаксическую ошибку.
argc	int	read only	Число аргументов.
argv	char **	read only	Вектор аргументов.
clictx	void **	write only	Принимает адрес вновь выделенного контекста разбора. При неудаче он освобождается и обнуляется в NULL.

Принимает грамматику `verb [p1 p2 ... p8] [/qualifier[=value] ...]`. Имена глаголов и квалификаторов можно сокращать до любого однозначного префикса. Каждое значение проверяется по объявленному типу его параметра или квалификатора (см. типы `__cli$val_type` ниже). Полученный контекст потребляется `__cli$dispatch()` и `__cli$get_value()` и должен быть освобождён через `__cli$cleanup()`.

Глагол может быть вложенным: он может нести цепочку подглаголов, так что `SHOW STATISTICS` разрешает `STATISTICS` в списке подглаголов `SHOW`. Подглагол следует за своим глаголом непосредственно - квалификатор между ними стоять не может.

Квалификатор пишется как `/name` (VMS) или `--name` (длинная форма GNU); одиночный `-` не является префиксом квалификатора, поэтому значение может начинаться с него. Параметры позиционны и задаются перед квалификаторами. Токен, чьё имя совпадает с определённым квалификатором, останавливает набор позиционных и берётся как этот квалификатор, так что опциональный параметр можно опустить в его пользу - `/etc/passwd`, будучи длиннее любого имени квалификатора, им не является и сохраняется как значение параметра, тогда как `/csv` - квалификатор. Голый токен, не занявший ни параметр, ни квалификатор, сообщается (`Unrecognized token`), а не отбрасывается молча; объявляйте параметр `CLI$M_REQUIRED` там, где его отсутствие само по себе должно быть ошибкой.

Условные значения

STS$K_SUCCESS	Командная строка разобрана без ошибок.
STS$K_ERROR	Значение не прошло проверку типа, либо не удалось выделить память.
STS$K_FATAL	Нераспознанный или неоднозначный глагол/квалификатор, либо слишком мало аргументов.

__cli$dispatch

Рутина Dispatch Command вызывает рутину-действие, связанную с разобранным глаголом.

Формат `int __cli$dispatch (clictx)`

Возврат `int` (write only) — Код-состояние, возвращённый рутиной-действием.

Параметр	Тип	Доступ	Описание
clictx	CLI_CTX *	read only	Адрес контекста, созданного __cli$parse().

Рутина-действие, зарегистрированная в совпавшем CLI_VERB, вызывается с контекстом и полем act_arg глагола. Внутри рутины-действия значения параметров и квалификаторов извлекаются через __cli$get_value().

Условные значения

STS$K_SUCCESS	Рутина-действие завершилась успешно.
(other)	Любой код-состояние, который вернёт рутина-действие.

__cli$get_value

Рутина Get Parameter or Qualifier Value извлекает строку значения, связанную с заданным дескриптором параметра или квалификатора.

Формат `int __cli$get_value (clictx, pq, val)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
clictx	CLI_CTX *	read only	Адрес контекста разбора.
pq	CLI_PQDESC *	read only	Адрес дескриптора параметра или квалификатора, значение которого требуется.
val	ASC *	write only	Принимает значение как ASCIC-строку.

Вызывается после __cli$parse(). Для квалификатора со списком (CLI$M_LIST, напр. /Q=(a,b,c)) последовательные вызовы возвращают значения по очереди, затем сообщают об отсутствии. Если элемент не задан, а его дескриптор содержит непустое поле defval, возвращается это значение по умолчанию.

Условные значения

STS$K_SUCCESS	Значение возвращено.
STS$K_WARN	Параметр или квалификатор отсутствовал в командной строке.

__cli$present

Рутина проверки присутствия сообщает, был ли задан параметр или квалификатор в командной строке и был ли он указан в отрицательной форме.

Формат `int __cli$present (CLI_CTX *clictx, CLI_PQDESC *pq, unsigned *state)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
clictx	CLI_CTX *	read only	Адрес контекста разбора.
pq	CLI_PQDESC *	read only	Дескриптор проверяемого параметра или квалификатора.
state	unsigned *	write only	Необязательный; принимает состояние элемента - CLI$M_PRESENT, плюс CLI$M_NEGATED, если квалификатор задан как /NO<name>. Ноль, если элемент отсутствует. Если квалификатор повторён - характернее всего /HIGHLIGHT, затем /NOHIGHLIGHT - выигрывает последнее вхождение в командной строке, как в DCL.

Аналог службы OpenVMS CLI$PRESENT. Дополняет `__cli$get_value()` для отрицаемого квалификатора (`CLI$M_NEGATABLE`): утвердительная и отрицательная формы различаются битом `CLI$M_NEGATED` в **state**, а не значением.

Если квалификатор задан несколько раз, сообщается состояние **последнего** вхождения: завершающее `/NO<name>` перекрывает более раннюю утвердительную форму и наоборот.

Условные значения

STS$K_SUCCESS	Параметр или квалификатор присутствует.
STS$K_WARN	Отсутствует.

__cli$cleanup

Рутина Release Parse Context освобождает всю память, принадлежащую контексту разбора.

Формат `int __cli$cleanup (clictx)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
clictx	CLI_CTX *	modify	Адрес освобождаемого контекста. Аргумент NULL принимается и игнорируется.

Должна вызываться, когда разобранные значения больше не нужны. После вызова указатель на контекст использовать нельзя.

Условные значения

STS$K_SUCCESS	Рутина успешно завершена.

__cli$show_verbs

Рутина Display Command Definitions записывает в журнал человекочитаемый список таблицы определения команд.

Формат `void __cli$show_verbs (verbs, level)`

Возврат Нет.

Параметр	Тип	Доступ	Описание
verbs	CLI_VERB *	read only	Адрес таблицы команд.
level	int	read only	Начальный уровень отступа; передайте 0 для таблицы верхнего уровня.

Предназначена для документирования или отладки грамматики команд; цепочки подглаголов выводятся рекурсивно с нарастающим отступом.

__cli$help

Рутина генерации справки печатает экран использования по таблицам команд: список команд либо SYNOPSIS глагола с его параметрами и квалификаторами.

Формат `int __cli$help (CLI_VERB *verbs, CLI_VERB *verb)`

Возврат `int` (write only) — STS$K_SUCCESS.

Параметр	Тип	Доступ	Описание
verbs	CLI_VERB *	read only	Таблица команд верхнего уровня; используется для перечисления команд, когда равен NULL.
verb	CLI_VERB *	read only	Описываемый глагол, либо NULL для обзора всех команд.

Пишет текст-подсказку в приёмник журнала (без префикса). Список параметров показывает каждый P*n* с его типом; список квалификаторов - каждое имя с типом, значением по умолчанию и атрибутами `required`/`/NO`/`list`.

Встроенная подсказка запрашивается через `/HELP`, `--help` или короткое `-h`. Ведущий `/` или `-` обязателен, поэтому голое `help` - это обычный глагол или значение параметра, а не запрос экрана. Короткое `-h` сохранено как намеренный алиас, хотя прочие Unix-квалификаторы используют форму `--`. Запрос обнаруживается `__cli$parse()` до проверки параметров; показывается нужная подсказка, и разбор возвращает `STS$K_WARN`, так что вызывающий пропускает `__cli$dispatch()`.

Условные значения

`STS$K_SUCCESS` Справка напечатана.

Глава 4. Справочник служб времени и FAO (starlet)

Эта глава описывает модуль `starlet` - переносимые аналоги служб времени и форматирования OpenVMS. Движок FAO пишет в дескриптор `fao_desc_t` (пара длина/указатель; длина - ёмкость буфера, за которую запись не выходит). Каждый аргумент директивы - `fao_prm_t` (машинное слово). Двоичные времена - значения `fao_time_t`: секунды эпохи UNIX для абсолютного времени, отрицательный счётчик для delta-времени. Основные директивы FAO:

`!AS !AZ !AC`	Строка по дескриптору / нуль-терм. / со счётчиком.
`!AD !AF`	Длина и адрес; !AF показывает непечатаемые точками.
`!UB !UW !UL !UQ`	Беззнаковое десятичное, 8/16/32/64 бит.
`!SB ... !SQ`	Знаковое десятичное.
`!XB ... !XQ`	Шестнадцатеричное.
`!OB ... !OQ`	Восьмеричное.
`!ZB ... !ZQ`	Десятичное с ведущими нулями.
`!%D !%T !%S`	Дата / дата-время / время по указателю fao_time_t.
`!n*c`	Повторить символ c n раз.
`!n(...!)`	Повторить группу директив n раз.
`!n<...!>`	Поле фиксированной ширины в n колонок.
`!/ !_ !^ !!`	Перевод строки, табуляция, прогон, литерал "!".

Директивы quadword (`!xQ`) и времени (`!%D`/`!%T`) передаются *по значению*: тип `fao_prm_t` 64-битный, поэтому quadword или время помещаются прямо в параметр на любой платформе. Все рутины этого модуля носят префикс `__util$`. FAO-рутины (`__util$fao`/`__util$faol`) заменили старый FAO из `utility_routines`.

__util$fao

Рутина Formatted ASCII Output форматирует список переменных аргументов в буфер вызывающего под управлением FAO-строки в стиле VMS.

Формат `int __util$fao (ctlstr, outlen, outbuf [, prm, ...])`

Возврат `int` (write only) — Код-состояние: `STS$K_SUCCESS`, либо `FAO$K_BUFOVF`, если результат усечён.

Параметр	Тип	Доступ	Описание
`ctlstr`	`const char *`	read only	Нуль-терминированная FAO-строка. Директивы вводятся символом `!`; см. таблицу в начале главы.
`outlen`	`unsigned short *`	write only	Принимает число байт, записанных в буфер. Может быть `NULL`.
`outbuf`	`fao_desc_t *`	modify	Дескриптор выходного буфера. `dsc$w_length` задаёт ёмкость на входе; запись за границу буфера не производится.
`prm`	`fao_prm_t`	read only	Аргументы директив. Каждый должен быть приведён к `fao_prm_t`. Директивы quadword (`!xQ`) и времени (`!%T`/`!%D`) передаются ПО ЗНАЧЕНИЮ (тип `fao_prm_t` 64-битный).

Переносимый аналог OpenVMS `SYS$FAO`. Вывод ограничен ёмкостью дескриптора; при переполнении результат усекается и возвращается `FAO$K_BUFOVF`. До `FAO$K_MAXPRM` аргументов кэшируется, чтобы работали директивы повторного использования параметров (`!n-`).

Условные значения

`STS$K_SUCCESS`	Строка отформатирована.
`FAO$K_BUFOVF`	Выходной буфер слишком мал; результат усечён.
`FAO$K_BADDIR`	Встречена некорректная директива.
`STS$K_ERROR`	Обязательный аргумент был `NULL`.

__util$faol

Рутина Formatted ASCII Output with List идентична __util$fao, за исключением того, что аргументы директив передаются явным массивом.

Формат `int __util$faol (ctlstr, outlen, outbuf, prmlst, prmcnt)`

Возврат `int` (write only) — Код-состояние, как для `__util$fao`.

Параметр	Тип	Доступ	Описание
`ctlstr`	`const char *`	read only	FAO-строка.
`outlen`	`unsigned short *`	write only	Принимает число записанных байт; может быть `NULL`.
`outbuf`	`fao_desc_t *`	modify	Дескриптор выходного буфера.
`prmlst`	`const fao_prm_t *`	read only	Массив аргументов директив.
`prmcnt`	`int`	read only	Число элементов в **prmlst**. Запросы за пределами массива дают ноль.

Переносимый аналог OpenVMS `SYS$FAOL`. Предпочтителен, когда число аргументов определяется во время выполнения.

Условные значения

`STS$K_SUCCESS`	Строка отформатирована.
`FAO$K_BUFOVF`	Результат усечён.
`FAO$K_BADDIR`	Встречена некорректная директива.

__util$bintim

Рутина Convert ASCII Time to Binary разбирает строку абсолютного или delta-времени в стиле OpenVMS в двоичное значение времени.

Формат `int __util$bintim (timstr, timadr)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`timstr`	`const char *`	read only	Нуль-терминированная строка времени. Абсолютная: `[dd]-MMM-[yyyy] [hh:mm:ss[.cc]]` либо только время суток. Delta: `dddd hh:mm:ss` - счётчик суток и время суток.
`timadr`	`fao_time_t *`	write only	Принимает двоичное время: секунды эпохи UNIX для абсолютного времени, отрицательный счётчик секунд для delta-времени.

Переносимый аналог OpenVMS `SYS$BINTIM`. Опущенные поля абсолютного времени берутся из текущей даты. День проверяется по длине месяца с учётом високосных лет. По соглашению delta-времена представляются отрицательными значениями - именно этого ожидают `__util$add_times` и `__util$sub_times`.

Условные значения

`STS$K_SUCCESS`	Строка разобрана.
`FAO$K_IVTIME`	Строка не является допустимым временем.
`STS$K_ERROR`	Передан аргумент `NULL`.

__util$gettim

Рутина Get System Time возвращает текущее время как двоичное значение.

Формат `int __util$gettim (timadr)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`timadr`	`fao_time_t *`	write only	Принимает текущее абсолютное время (секунды эпохи UNIX).

Переносимый аналог OpenVMS `SYS$GETTIM`. Источник времени по умолчанию - хостовая C-библиотека; его можно переопределить при сборке через макрос `FAO$GETTIME`.

Условные значения

`STS$K_SUCCESS`	Рутина успешно завершена.
`STS$K_ERROR`	Передан аргумент `NULL`.

__util$asctim

Рутина Convert Binary Time to ASCII форматирует двоичное значение времени в читаемую строку.

Формат `int __util$asctim (outlen, outbuf, timadr, cvtflg)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
`outlen`	`unsigned short *`	write only	Принимает число записанных байт; может быть `NULL`.
`outbuf`	`fao_desc_t *`	modify	Дескриптор выходного буфера.
`timadr`	`const fao_time_t *`	read only	Адрес форматируемого двоичного времени.
`cvtflg`	`int`	read only	Флаг преобразования: 0 форматирует полное `dd-MMM-yyyy hh:mm:ss.cc`, ненулевое - только время суток.

Переносимый аналог OpenVMS `SYS$ASCTIM`. Вывод ограничен ёмкостью дескриптора; значение, полученное от `__util$bintim`, воспроизводится в точности.

Условные значения

`STS$K_SUCCESS`	Время отформатировано.
`FAO$K_BUFOVF`	Выходной буфер слишком мал; результат усечён.
`STS$K_ERROR`	Передан аргумент `NULL`.

__util$add_times

Рутина Add Two Binary Times складывает два двоичных времени по правилам комбинирования OpenVMS для абсолютных и delta-времён.

Формат `int __util$add_times (tim1, tim2, result)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
tim1	const fao_time_t *	read only	Первый операнд.
tim2	const fao_time_t *	read only	Второй операнд.
result	fao_time_t *	write only	Принимает сумму.

Переносимый аналог OpenVMS `LIB$ADD_TIMES`. Delta-времена отрицательны, абсолютные - неотрицательны. Два delta дают delta; абсолютное и delta дают абсолютное, сдвинутое на величину delta; два абсолютных отвергаются.

Условные значения

STS$K_SUCCESS	Сложение выполнено.
FAO$K_IVTIME	Оба операнда - абсолютные времена.
STS$K_ERROR	Передан аргумент NULL.

__util$sub_times

Рутина Subtract Two Binary Times вычитает второе двоичное время из первого по правилам комбинирования OpenVMS.

Формат `int __util$sub_times (tim1, tim2, result)`

Возврат `int` (write only) — Код-состояние.

Параметр	Тип	Доступ	Описание
tim1	const fao_time_t *	read only	Уменьшаемое.
tim2	const fao_time_t *	read only	Вычитаемое.
result	fao_time_t *	write only	Принимает разность.

Переносимый аналог OpenVMS `LIB$SUB_TIMES`. Абсолютное минус абсолютное даёт delta (и **tim1** не должно предшествовать **tim2**); абсолютное минус delta даёт абсолютное; delta минус delta даёт delta; delta минус абсолютное отвергается.

Условные значения

STS$K_SUCCESS	Вычитание выполнено.
FAO$K_NEGTIM	Результат был бы отрицательным.
FAO$K_IVTIME	Запрошено вычитание delta минус абсолютное.
STS$K_ERROR	Передан аргумент NULL.

Глава 5. Примеры программирования

Следующие законченные программы показывают, как рутины сочетаются на практике. Каждая была скомпилирована с модулями и запущена; показанный вывод - настоящий результат (префикс метки времени и pid у строк журнала опущен для краткости).

Пример A.1. Первое приложение: опции, конфиг и журнал

Костяк программы: объявить таблицу опций, разобрать командную строку и конфиг одной таблицей, писать в журнал.

```
/*
**  Пример A.1 — Первое приложение: опции, конфиг и журнал
**
**  Файл:  app.c
**  Собрать:  cc -std=gnu11 -O2 -I. app.c utility_routines.c -o app
**  Запуск:  ./app
*/
```

```
#define __MODULE__ "APP$"          /* имя модуля печатается в каждой строке журнала */
#include "utility_routines.h"

int main(int argc, char **argv)
{
int             l_trace = 0;
unsigned short  l_port  = 0;
ASC             l_logfile = {0}, l_config = {0};

    /* Таблица опций: имя, приёмник значения, размер приёмника, тип. */
    /* OPTS_NULL - признак конца таблицы.                            */
    const OPTS l_opts[] = {
        { $ASCINI("config"),  &l_config,  ASC$K_SZ, OPTS$K_CONF },
        { $ASCINI("port"),    &l_port,    0,        OPTS$K_PORT },
        { $ASCINI("logfile"), &l_logfile, ASC$K_SZ, OPTS$K_STR  },
        { $ASCINI("trace"),   &l_trace,   0,        OPTS$K_OPT  },
        OPTS_NULL
    };

    /* Разобрать argv. Опция -config= сразу читает конфиг-файл той же таблицей. */
    /* Проверка (1 & status): младший бит статуса равен 1 при успехе.          */
    if ( !(1 & __util$getparams(argc, argv, (OPTS *) l_opts)) )
        return  $LOG(STS$K_FATAL, "invalid command line");

    /* Если задан лог-файл - перенаправить журнал туда */
    if ( $ASCLEN(&l_logfile) )
        {
        char l_path[256];
        __util$asc2str(&l_logfile, l_path);
        __util$deflog(l_path, NULL);
        }

    __util$showparams(l_opts);
    $LOG(STS$K_INFO, "app started on port %d", l_port);   /* STS$K_INFO -> буква I */
    return  STS$K_SUCCESS;
}
```

Вывод

```
%UTILS-I:  trace = OFF
%APP-I:  app started on port 0, trace=0
( a real invocation: app -config=app.conf -port=8080 -trace )
```

Пример A.2. Работа со временем (starlet): разбор, арифметика, формат

Превратить строку времени в число, прибавить интервал, получить строку обратно. Правило: абсолютное время ≥ 0, а delta (интервал) хранится как отрицательное число.

```
/*
**  Пример A.2 — Работа со временем (starlet): разбор, арифметика, формат
**
**  Файл:  tim.c
**  Собрать:  cc -std=gnu11 -O2 -I. tim.c starlet.c -o tim
**  Запуск:  ./tim
*/
```

```
#include <stdio.h>
#include "utility_routines.h"
#include "starlet.h"

int main(void)
{
fao_time_t     l_t, l_delta, l_later;
char           l_buf [40];
fao_desc_t     l_dsc = $FAO_DESC(l_buf);      /* дескриптор хранит размер буфера */
unsigned short l_len;

    /* Абсолютное время (момент) -> секунды эпохи; получится 1539616633 */
    __util$bintim("15-OCT-2018 15:17:13", &l_t);

    /* Интервал "1 сутки 12 часов". Формат "дни ч:м:с".            */
    /* delta хранится как ОТРИЦАТЕЛЬНОЕ число: получится -129600.   */
    __util$bintim("1 12:00:00", &l_delta);

    /* Момент + delta = новый момент, сдвинутый на 1.5 суток вперёд */
    __util$add_times(&l_t, &l_delta, &l_later);

    /* Число -> строка. Флаг 0 = "дата и время". Буфер не переполнится - */
    /* его размер уже записан в дескриптор макросом $FAO_DESC.            */
    __util$asctim(&l_len, &l_dsc, &l_later, 0);
    l_buf[l_len] = '\0';

    printf("t+delta = %s\n", l_buf);
    return 0;
}
```

Вывод

```
t+delta = 17-OCT-2018 03:17:13.00

( 15-OCT 15:17 + 1.5 суток = 17-OCT 03:17 )
```

Пример A.3. Контроль целостности: CRC-32 и hex

Посчитать контрольную сумму буфера и напечатать его в шестнадцатеричном виде.

```
/*
**  Пример A.3 — Контроль целостности: CRC-32 и hex
**
**  Файл:  crc.c
**  Собрать:  cc -std=gnu11 -O2 -I. crc.c utility_routines.c -o crc
**  Запуск:  ./crc
*/
```

```
#define __MODULE__ "CRC$"
#include "utility_routines.h"

int main(void)
{
unsigned char  l_frame[] = { 0x02, 0x10, 0x00, 0x04, 0xDE, 0xAD, 0xBE, 0xEF };
unsigned       l_crc;
char           l_hex[2 * sizeof(l_frame) + 1];

    /* CRC-32 по всему массиву. 0 - начальное значение (новая сумма).      */
    /* Для потока: результат передают начальным значением следующего вызова */
    l_crc = __util$crc32c(0, l_frame, sizeof(l_frame));

    /* Байты -> hex-строка (байт = две буквы). Приёмник: 2*N + 1 байт. */
    __util$bin2hex(l_frame, l_hex, sizeof(l_frame));

    /* Имя crc32c историческое: считается стандартный CRC-32 (как в zlib) */
    $LOG(STS$K_INFO, "frame [%s] crc32=0x%08x", l_hex, l_crc);
    return  STS$K_SUCCESS;
}
```

Вывод

```
frame [02100004deadbeef] crc32=0xe8819e5c
```

Пример A.4. Очередь заданий

Простейший конвейер "производитель-потребитель" на двусвязной очереди.

```
/*
**  Пример A.4 — Очередь заданий
**
**  Файл:  que.c
**  Собрать:  cc -std=gnu11 -O2 -I. que.c utility_routines.c -o que
**  Запуск:  ./que
*/
```

```
#define __MODULE__ "QUE$"
#include "utility_routines.h"

/* У элемента очереди ПЕРВЫМ полем обязан идти ENTRY - пара указателей, */
/* которыми очередь связывает элементы.                                  */
typedef struct { ENTRY links; int value; } WORK_ITEM;

int main(void)
{
__QUEUE    l_q = QUEUE_INITIALIZER;
WORK_ITEM  l_pool[8] = {0};        /* = {0} обязательно: вставка проверяет служебное
поле */
unsigned   l_count;
WORK_ITEM *l_item;

    /* Положить 8 элементов в хвост. В l_count - число элементов до вставки. */
    for ( int i = 0; i < 8; i++ )
        {
        l_pool[i].value = i * 100;
        $INSQTAIL(&l_q, &l_pool[i], &l_count);
        }

    /* Достаём с головы (FIFO). $REMQHEAD возвращает "успех" даже на пустой  */
    /* очереди, поэтому страж && l_count ОБЯЗАТЕЛЕН - иначе цикл не остановится. */
    while ( (1 & $REMQHEAD(&l_q, &l_item, &l_count)) && l_count )
        $LOG(STS$K_INFO, "value=%d", l_item->value);
    return  STS$K_SUCCESS;
}
```

Вывод

```
%UTILS-I:  processing item value=0
%UTILS-I:  processing item value=100
%UTILS-I:  processing item value=200
...  (through value=700)
```

Пример A.5. Разбор командной строки (CLI): глагол, параметры, квалификаторы

Описать команду `COPY source dest /verbose /port=8080` и разобрать её. Команда = глагол + позиционные параметры P1..P8 + квалификаторы `/имя[=значение]`.

```
/*
**  Пример A.5 — Разбор командной строки (CLI): глагол, параметры, квалификаторы
**
**  Файл:  cli.c
**  Собрать:  cc -std=gnu11 -O2 -I. cli.c cli_routines.c utility_routines.c -o cli
**  Запуск:  ./cli
*/
```

```
#define __MODULE__ "DEMO$"
#include "utility_routines.h"
#include "cli_routines.h"

static CLI_PQDESC copy_params[];

/* Действие, вызываемое для глагола COPY */
static int copy_action(CLI_CTX *a_ctx, void *a_arg)
{
ASC l_src = {0};

    (void) a_arg;
    /* Достать значение P1 (source) из разобранного контекста */
    if ( 1 & __cli$get_value(a_ctx, &copy_params[0], &l_src) )
        $LOG(STS$K_INFO, "COPY source = %.*s", $ASC(&l_src));
    return STS$K_SUCCESS;
}

/* P1 = source, P2 = dest (оба - имена файлов) */
/* Позиционные параметры: source = P1, dest = P2; тип FILE = имя файла */
static CLI_PQDESC copy_params[] = {
    { .name = {$ASCINI("source")}, .type = CLI$K_FILE, .pn = CLI$K_P1 },
    { .name = {$ASCINI("dest")},   .type = CLI$K_FILE, .pn = CLI$K_P2 },
    { {0} }
};
/* /VERBOSE (флаг), /PORT=<число> */
/* Квалификаторы: /verbose - флаг (OPT), /port - число (NUM, проверяется по типу) */
static CLI_PQDESC copy_quals[] = {
    { .name = {$ASCINI("verbose")}, .type = CLI$K_OPT },
    { .name = {$ASCINI("port")},    .type = CLI$K_NUM },
    { {0} }
};
/* Таблица глаголов: связывает "copy" с параметрами, квалификаторами и действием */
static CLI_VERB commands[] = {
    { .name = {$ASCINI("copy")}, .params = copy_params, .quals = copy_quals, .act_rtn =
copy_action },
    { {0} }
};

int main(int argc, char **argv)
{
void *l_ctx = NULL;

    /* Разобрать argv. CLI$M_OPSIGNAL - печатать сообщение при ошибке.       */
    if ( !(1 & __cli$parse(commands, CLI$M_OPSIGNAL, argc-1, argv+1, &l_ctx)) )
        return  STS$K_ERROR;

    __cli$dispatch(l_ctx);   /* вызвать copy_action; внутри - __cli$get_value */
    __cli$cleanup(l_ctx);
```

```
    return  STS$K_SUCCESS;
}
```

Вывод

```
$ COPY a.txt b.txt /verbose /port=8080
%UTILS-I:  COPY source = a.txt

$ COPY a.txt b.txt /port=abc
%CLI_RTNS-E:  Value 'abc' is not a valid number   (отвергнуто)
```

Пример A.6. Бинарный протокол avproto: сборка и разбор пакета

Собрать пакет из записей Tag-Length-Value и разобрать его на другой стороне. Все числа хранятся в сетевом порядке байт.

```
/*
**  Пример A.6 — Бинарный протокол avproto: сборка и разбор пакета
**
**  Файл:  avp.c
**  Собрать:  cc -std=gnu11 -O2 -I. avp.c avproto.c utility_routines.c -o avp
**  Запуск:  ./avp
*/
```

```
#define __MODULE__ "AVP$"
#define __FAC__    "AVP"
#include <stdint.h>
#include <arpa/inet.h>
#include "utility_routines.h"
#include "avproto.h"

#define APP$K_TAG_ID    10
#define APP$K_TAG_NAME  11

/* Собрать запрос: вернуть длину готового PDU */
static unsigned build_request (void *a_pdu, unsigned a_pdusz)
{
AVPROTO_PDU *l_pdu = (AVPROTO_PDU *) a_pdu;
uint32_t     l_id  = 0x1234ABCD;

    /* Заголовок: 8-байтная сигнатура (ровно 8, без нулей внутри!), csr, seq. */
    /* hset обнуляет длину, поэтому вызывается ДО первого __tlv$put.          */
    __tlv$hset(l_pdu, (const unsigned char *) "TLV-PDU1", 1, 42);
    /* Запись TLV: тег, тип, значение. Здесь - 32-битное число, затем блок байт */
    __tlv$put(l_pdu, a_pdusz, APP$K_TAG_ID, TAG$K_LONGWORD, &l_id, 4);
    __tlv$put(l_pdu, a_pdusz, APP$K_TAG_NAME, TAG$K_BBLOCK,   "demo-c", 6);

    return  sizeof(AVPROTO_HDR) + ntohl(l_pdu->r_hdr.u_len);
}

/* Разобрать принятый PDU */
static int parse_request (void *a_pdu, unsigned a_rcvlen)
{
unsigned  l_ulen, l_ucsr, l_useq, l_vtype, l_vsz;
uint32_t  l_id;
char      l_name[64];
int       l_ctx;

    /* Проверить сигнатуру и извлечь поля заголовка */
    if ( !(1 & __tlv$hget(a_pdu, (const unsigned char *) "TLV-PDU1", &l_ulen, &l_ucsr,
&l_useq)) )
        return  $LOG(STS$K_ERROR, "foreign or corrupted PDU");

    /* Longword по тегу APP$K_TAG_ID */
    l_ctx = -1; l_vsz = sizeof(l_id);
    /* a_rcvlen - РЕАЛЬНО принятый размер: по нему проверяется каждая запись, */
    /* чтобы битый пакет не увёл за границы буфера.                            */
    if ( 1 & __tlv$get(a_pdu, a_rcvlen, &l_ctx, APP$K_TAG_ID, &l_vtype, &l_id,
&l_vsz) )
        $LOG(STS$K_INFO, "id = 0x%08x", l_id);

    /* Блок по тегу APP$K_TAG_NAME (valsz обязателен для переменной длины) */
    l_ctx = -1; l_vsz = sizeof(l_name);
```

```
    if ( 1 & __tlv$get(a_pdu, a_rcvlen, &l_ctx, APP$K_TAG_NAME, &l_vtype, l_name,
&l_vsz) )
        $LOG(STS$K_INFO, "name = %.*s", l_vsz, l_name);

    return  STS$K_SUCCESS;
}

int main(void)
{
char     l_buf[8192];
unsigned l_len;

    l_len = build_request(l_buf, sizeof(l_buf));     /* «отправитель» */
    $LOG(STS$K_INFO, "built PDU of %u octets", l_len);

    parse_request(l_buf, l_len);                     /* «получатель» (l_len - реальный
размер) */
    __tlv$dump(l_buf, l_len);
    return  STS$K_SUCCESS;
}
```

Вывод

```
%AVP-I:  built PDU of 38 octets
%AVP-I:  id = 0x1234abcd
%AVP-I:  name = demo-c
%UTILS-I:  PDU [len=18, csr=0x1, seq=42]
%UTILS-I:  [0000] TLV [tag=400a(id=000a, type=0004), len=04] 0x400a0004:0x1234abcd
%UTILS-I:  [0001] TLV [tag=000b(id=000b, type=0000), len=06] 0x000b0006:0x64656d6f2d63
```

Приложение A. Справочник директив FAO

Служба FAO модуля `starlet` формирует управляющую строку со встроенными директивами. Каждый параметр передаётся как 64-битный `fao_prm_t`; указатель передаётся как `(fao_prm_t)(uintptr_t) ptr`. Макросы `$PUTMSG_FAO()` и `$LOG_FAO()` выполняют приведение и выводят счётчик параметров за вызывающего.

A.1 Сводка директив

Директива	Описание
`!AS`	Строка по дескриптору (fao_desc_t *).
`!AZ`	Строка, завершённая нулём (char *).
`!AC`	Строка со счётчиком, первый байт — длина.
`!AD`	Два параметра: длина, адрес.
`!AF`	Как !AD, непечатаемые заменяются '.'.
`!UB !UW !UL !UQ`	Беззнаковое десятичное, 8/16/32/64 бита.
`!SB !SW !SL !SQ`	Знаковое десятичное.
`!XB !XW !XL !XQ`	Шестнадцатеричное, с нулями до 2/4/8/16 цифр.
`!OB !OW !OL !OQ`	Восьмеричное, с ведущими нулями.
`!ZB !ZW !ZL !ZQ`	Беззнаковое десятичное с ведущими нулями.
`!/ !_ !^ !!`	CR+LF; табуляция; прогон страницы; литерал '!'.
`!%S`	's', если последнее преобразованное число не равно 1.
`!%T`	Время, hh:mm:ss.cc (UTC + FAO$K_TZOFFSET).
`!%D`	Дата, dd-MMM-yyyy hh:mm:ss.cc.
`!%M`	MAC / аппаратный адрес, aa:bb:cc:dd:ee:ff — указатель на 6 октетов.
`!%I`	Адрес IPv4, d.d.d.d — указатель на 4 октета.
`!%6`	IPv6, каноническая форма RFC 5952; IPv4-mapped — в смешанной записи ::ffff:a.b.c.d — указатель на 16 октетов.
`!%U`	UUID, 8-4-4-4-12 — указатель на 16 октетов.
`!nXL ...`	Явная ширина поля вывода директивы.
`!n*c`	Повторить символ c n раз.
`!n(...)`	Повторить группу директив n раз.
`!n<...!>`	Поле фиксированной ширины: дополняется пробелами или усекается до n колонок.
`!n-`	Использовать последние n параметров повторно.
`!n+`	Пропустить следующие n параметров.

ПРИМЕЧАНИЕ
Адресные директивы принимают УКАЗАТЕЛЬ на октеты (сетевой порядок); нулевой указатель даёт `(null)`. Форма IPv6 следует RFC 5952: самая длинная серия из двух и

более нулевых групп сжимается в `::` (при равенстве — левая), одиночная нулевая группа не сжимается.

Приложение B. Дополнения после основного корпуса

B.1 $LOG_FAO — журналирование по FAO-строке

```
$LOG_FAO(severity, ctl, ...)
```

FAO-близнец `$LOG`: тело — управляющая FAO-строка, которую формирует `__util$faol()`, так что доступна каждая директива Приложения A, включая адресные. Макрос сам приводит каждый параметр к `fao_prm_t` и сам выводит счётчик — эргономика `$PUTMSG_FAO`; форма без параметров допустима. Запись несёт обычный конверт «метка времени / `%<фасилити>-<S>:`» (в отладочной сборке — ещё и `[модуль\функция:строка]`); `STS$M_SYSLOG` отправляет уже отформатированное тело в syslog. За макросом стоят рутины `__util$log_fao()` и `__util$logd_fao()`.

```
$LOG_FAO(STS$K_INFO, "if=!AZ rx=!UL peer=!%I", "eth0", 777, l_ip4);
  /* 29-08-2026 17:12:10.013   765 [MOD\main:47] %TFAC-I:  if=eth0 rx=777
peer=192.168.1.10 */
```

B.2 Сопоставление с шаблоном — __util$pattern_match()

Сопоставитель итеративный, с одной точкой бэктрека, ограничен O(|строка| × |шаблон|) — враждебные многозвёздочные шаблоны его не взрывают. Подстановочные знаки:

Знак	Значение
*	Любая серия октетов, включая пустую.
%	Ровно один октет.
?	Алиас %, ровно один октет (DOS/glob-написание). Литеральный вопросительный знак в шаблоне отныне — подстановочный.

B.3 Имя фасилити живёт в записи

`EMSG_RECORD` несёт хвостовое поле `facname`, которое `__util$inimsg()` штампует в каждую запись при регистрации; `__util$putmsg_fao()`/`__util$putmsgd_fao()` читают имя из найденной записи — без второго прохода по списку дескрипторов на каждое сообщение. NULL или пустое имя отображается как `NONAME`. Существующие позиционные инициализаторы записей остаются в силе: поле хвостовое и заполняется библиотекой, а не вызывающим.

B.4 Префикс лог-записи

Префикс «DD-MM-YYYY HH:MM:SS.msc <tid> » формирует кэширующий хелпер: датно-временная часть пересчитывается только при смене секунды, а идентификатор нити берётся у ядра один раз на нить. Журналирование целиком стало примерно вчетверо быстрее; байты идентичны историческому виду.

ПРИМЕЧАНИЕ
После fork() потомок наследует кэшированный идентификатор нити родителя; процесс, который делает fork() после первой лог-записи и которому важна колонка tid, должен сбросить кэш через pthread_atfork() либо перезапуститься через exec.